Lab 6

23BCE9320

QUESTION 1:

Use MongoDB to implement the following DB operations

1.Create a database called ‘vehicles’ and write a MongoDB query to select database as“vehicles”.

```
C:\Users\23bce9320\Desktop>mongosh
Current Mongosh Log ID: 69b137cf8916d9531f7c2906
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000
&appName=mongosh+2.7.0
Using MongoDB:          5.0.3
Using Mongosh:          2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-03-11T14:04:27.570+05:30: Access control is not enabled for the database. Read and write acces
s to data and configuration is unrestricted
------

Warning: Found ~/.mongorc.js, but not ~/.mongoshrc.js. ~/.mongorc.js will not be loaded.
  You may want to copy or rename ~/.mongorc.js to ~/.mongoshrc.js.
test> use vehicles
switched to db vehicles
```

2. Write a MongoDB query to display all the databases.

```
vehicles> show dbs
admin   40.00 KiB
config  96.00 KiB
local   80.00 KiB
```

3. Create a collection called ‘two_wheelers’. (use capping) and Create a collection called ‘four_wheelers’.

```
vehicles> // Capped collection for two_wheelers (size: 1MB, max 1000 documents)
| db.createCollection("two_wheelers", { capped: true, size: 1048576, max: 1000 })
{ ok: 1 }
vehicles> db.createCollection("four_wheelers")
{ ok: 1 }
```

4. Add 5 two-wheeler details to the collection named ‘two_wheelers’. Each document consists of following fields as bike_name, model (gear or gearless), category (100cc,125cc, 150cc, 200cc), colors_available (red, black, blue, sport red etc) as array,manufacturer, performance (out of 10), timestamp (date and year release) and price.

```
vehicles> db.two_wheelers.insertMany([
  {
    bike_name: "Pulsar NS200",
    model: "gear",
    category: "200cc",
    colors_available: ["red", "black", "blue"],
    manufacturer: "Bajaj",
    performance: 8.5,
    timestamp: ISODate("2024-03-15"),
    price: 145000
  },
  {
    bike_name: "Shine",
    model: "gearless",
    category: "125cc",
    colors_available: ["red", "black", "silver"],
    manufacturer: "Honda",
    performance: 7.8,
    timestamp: ISODate("2024-01-20"),
    price: 85000
  },
  {
    bike_name: "Apache RTR 160",
    model: "gear",
    category: "160cc",
    colors_available: ["sport red", "black", "blue"],
    manufacturer: "TVS",
    performance: 8.2,
    timestamp: ISODate("2024-02-10"),
    price: 125000
  },
  {
    bike_name: "Activa 6G",
    model: "gearless",
    category: "125cc",
    colors_available: ["matte black", "pearl white"],
    manufacturer: "Honda",
    performance: 7.5,
    timestamp: ISODate("2024-04-01"),
    price: 78000
  },
  {
    bike_name: "FZS FI V4",
    model: "gear",
    category: "150cc",
    colors_available: ["dark blue", "racing blue"],
    manufacturer: "Yamaha",
    performance: 8.0,
    timestamp: ISODate("2024-03-01"),
    price: 135000
```

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b138008916d9531f7c2907'),
    '1': ObjectId('69b138008916d9531f7c2908'),
    '2': ObjectId('69b138008916d9531f7c2909'),
    '3': ObjectId('69b138008916d9531f7c290a'),
    '4': ObjectId('69b138008916d9531f7c290b')
  }
}
```

5. Add 5 four-wheeler details to the collection named ‘four_wheelers’. Each document consists of following fields as vehicle_name, model (commercial or own), category(car, lorry, bus, mini truck, heavy truck, containers), variants (vxi, zxi, petrol, diesel etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

```
vehicles> db.four_wheelers.insertMany([
  {
    vehicle_name: "Swift",
    model: "own",
    category: "car",
    variants: ["vxi", "zxi", "petrol"],
    manufacturer: "Maruti Suzuki",
    performance: 7.9,
    timestamp: ISODate("2024-02-15"),
    price: 750000
  },
  {
    vehicle_name: "Eicher Pro 2049",
    model: "commercial",
    category: "mini truck",
    variants: ["diesel", "cng"],
    manufacturer: "Eicher",
    performance: 8.1,
    timestamp: ISODate("2024-01-10"),
    price: 1250000
  },
  {
    vehicle_name: "Creta",
    model: "own",
    category: "car",
    variants: ["petrol", "diesel", "turbo"],
    manufacturer: "Hyundai",
    performance: 8.4,
    timestamp: ISODate("2024-03-20"),
    price: 1150000
  },
  {
    vehicle_name: "Blaize",
    model: "commercial",
    category: "bus",
    variants: ["diesel"],
    manufacturer: "Tata",
    performance: 7.6,
    timestamp: ISODate("2024-02-05"),
    price: 2800000
  },
  {
    vehicle_name: "Innova Crysta",
    model: "own",
    category: "car",
    variants: ["diesel", "2.8L"],
    manufacturer: "Toyota",
    performance: 8.7,
    timestamp: ISODate("2024-04-10"),
    price: 2200000
```

```
]
)
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b138078916d9531f7c290c'),
    '1': ObjectId('69b138078916d9531f7c290d'),
    '2': ObjectId('69b138078916d9531f7c290e'),
    '3': ObjectId('69b138078916d9531f7c290f'),
    '4': ObjectId('69b138078916d9531f7c2910')
  }
}
```

6. Write a MongoDB query to display all documents available in two_wheelers and four_wheelers.

```
vehicles> db.two_wheelers.find().pretty()
[
  {
    _id: ObjectId('69b138008916d9531f7c2907'),
    bike_name: 'Pulsar NS200',
    model: 'gear',
    category: '200cc',
    colors_available: [ 'red', 'black', 'blue' ],
    manufacturer: 'Bajaj',
    performance: 8.5,
    timestamp: ISODate('2024-03-15T00:00:00.000Z'),
    price: 145000
  },
  {
    _id: ObjectId('69b138008916d9531f7c2908'),
    bike_name: 'Shine',
    model: 'gearless',
    category: '125cc',
    colors_available: [ 'red', 'black', 'silver' ],
    manufacturer: 'Honda',
    performance: 7.8,
    timestamp: ISODate('2024-01-20T00:00:00.000Z'),
    price: 85000
  },
  {
    _id: ObjectId('69b138008916d9531f7c2909'),
    bike_name: 'Apache RTR 160',
    model: 'gear',
    category: '160cc',
    colors_available: [ 'sport red', 'black', 'blue' ],
    manufacturer: 'TVS',
    performance: 8.2,
    timestamp: ISODate('2024-02-10T00:00:00.000Z'),
    price: 125000
  },
  {
    _id: ObjectId('69b138008916d9531f7c290a'),
    bike_name: 'Activa 6G',
    model: 'gearless',
    category: '125cc',
    colors_available: [ 'matte black', 'pearl white' ],
    manufacturer: 'Honda',
    performance: 7.5,
    timestamp: ISODate('2024-04-01T00:00:00.000Z'),
    price: 78000
  },
  {
    _id: ObjectId('69b138008916d9531f7c290b'),
    bike_name: 'FZS FI V4',
    model: 'gear',
```

```
    category: '150cc',
    colors_available: [ 'dark blue', 'racing blue' ],
    manufacturer: 'Yamaha',
    performance: 8,
    timestamp: ISODate('2024-03-01T00:00:00.000Z'),
    price: 135000
  }
]
vehicles> db.four_wheelers.find().pretty()
[
  {
    _id: ObjectId('69b138078916d9531f7c290c'),
    vehicle_name: 'Swift',
    model: 'own',
    category: 'car',
    variants: [ 'vxi', 'zxi', 'petrol' ],
    manufacturer: 'Maruti Suzuki',
    performance: 7.9,
    timestamp: ISODate('2024-02-15T00:00:00.000Z'),
    price: 750000
  },
  {
    _id: ObjectId('69b138078916d9531f7c290d'),
    vehicle_name: 'Eicher Pro 2049',
    model: 'commercial',
    category: 'mini truck',
    variants: [ 'diesel', 'cng' ],
    manufacturer: 'Eicher',
    performance: 8.1,
    timestamp: ISODate('2024-01-10T00:00:00.000Z'),
    price: 1250000
  },
  {
    _id: ObjectId('69b138078916d9531f7c290e'),
    vehicle_name: 'Creta',
    model: 'own',
    category: 'car',
    variants: [ 'petrol', 'diesel', 'turbo' ],
    manufacturer: 'Hyundai',
    performance: 8.4,
    timestamp: ISODate('2024-03-20T00:00:00.000Z'),
    price: 1150000
  },
  {
    _id: ObjectId('69b138078916d9531f7c290f'),
    vehicle_name: 'Blaize',
    model: 'commercial',
    category: 'bus',
    variants: [ 'diesel' ],
    manufacturer: 'Tata',
    performance: 7.6,
```

```
    performance: 7.6,
    timestamp: ISODate('2024-02-05T00:00:00.000Z'),
    price: 2800000
  },
  {
    _id: ObjectId('69b138078916d9531f7c2910'),
    vehicle_name: 'Innova Crysta',
    model: 'own',
    category: 'car',
    variants: [ 'diesel', '2.8L' ],
    manufacturer: 'Toyota',
    performance: 8.7,
    timestamp: ISODate('2024-04-10T00:00:00.000Z'),
    price: 2200000
  }
]
```

7. Write a MongoDB query to display only vehicle name and price in all the collection of the database

```
vehicles> db.two_wheelers.find({}, { bike_name: 1, price: 1, _id: 0 })
[
  { bike_name: 'Pulsar NS200', price: 145000 },
  { bike_name: 'Shine', price: 85000 },
  { bike_name: 'Apache RTR 160', price: 125000 },
  { bike_name: 'Activa 6G', price: 78000 },
  { bike_name: 'FZS FI V4', price: 135000 }
]
vehicles> db.four_wheelers.find({}, { vehicle_name: 1, price: 1, _id: 0 })
[
  { vehicle_name: 'Swift', price: 750000 },
  { vehicle_name: 'Eicher Pro 2049', price: 1250000 },
  { vehicle_name: 'Creta', price: 1150000 },
  { vehicle_name: 'Blaize', price: 2800000 },
  { vehicle_name: 'Innova Crysta', price: 2200000 }
]
```

8. Write a MongoDB query to display two_wheelers from a particular company

```
vehicles> db.two_wheelers.find({ manufacturer: "Honda" })
[
  {
    _id: ObjectId('69b138008916d9531f7c2908'),
    bike_name: 'Shine',
    model: 'gearless',
    category: '125cc',
    colors_available: [ 'red', 'black', 'silver' ],
    manufacturer: 'Honda',
    performance: 7.8,
    timestamp: ISODate('2024-01-20T00:00:00.000Z'),
    price: 85000
  },
  {
    _id: ObjectId('69b138008916d9531f7c290a'),
    bike_name: 'Activa 6G',
    model: 'gearless',
    category: '125cc',
    colors_available: [ 'matte black', 'pearl white' ],
    manufacturer: 'Honda',
    performance: 7.5,
    timestamp: ISODate('2024-04-01T00:00:00.000Z'),
    price: 78000
  }
]
```

9. Write a MongoDB query to display four_wheelers available in diesel variants

```
vehicles> db.four_wheelers.find({
|   "variants": "diesel"
| })
|
[
  {
    _id: ObjectId('69b138078916d9531f7c290d'),
    vehicle_name: 'Eicher Pro 2049',
    model: 'commercial',
    category: 'mini truck',
    variants: [ 'diesel', 'cng' ],
    manufacturer: 'Eicher',
    performance: 8.1,
    timestamp: ISODate('2024-01-10T00:00:00.000Z'),
    price: 1250000
  },
  {
    _id: ObjectId('69b138078916d9531f7c290e'),
    vehicle_name: 'Creta',
    model: 'own',
    category: 'car',
    variants: [ 'petrol', 'diesel', 'turbo' ],
    manufacturer: 'Hyundai',
    performance: 8.4,
    timestamp: ISODate('2024-03-20T00:00:00.000Z'),
    price: 1150000
  },
```

```
  {
    _id: ObjectId('69b138078916d9531f7c290f'),
    vehicle_name: 'Blaize',
    model: 'commercial',
    category: 'bus',
    variants: [ 'diesel' ],
    manufacturer: 'Tata',
    performance: 7.6,
    timestamp: ISODate('2024-02-05T00:00:00.000Z'),
    price: 2800000
  },
  {
    _id: ObjectId('69b138078916d9531f7c2910'),
    vehicle_name: 'Innova Crysta',
    model: 'own',
    category: 'car',
    variants: [ 'diesel', '2.8L' ],
    manufacturer: 'Toyota',
    performance: 8.7,
    timestamp: ISODate('2024-04-10T00:00:00.000Z'),
    price: 2200000
  }
```

10. Write a MongoDB query to display vehicles name, category and manufacturer details whose rating is more than 5.

```
vehicles> db.two_wheelers.find(
|   { performance: { $gt: 5 } },
|   { bike_name: 1, category: 1, manufacturer: 1, _id: 0 }
| )
[
  { bike_name: 'Pulsar NS200', category: '200cc', manufacturer: 'Bajaj' },
  { bike_name: 'Shine', category: '125cc', manufacturer: 'Honda' },
  { bike_name: 'Apache RTR 160', category: '160cc', manufacturer: 'TVS' },
  { bike_name: 'Activa 6G', category: '125cc', manufacturer: 'Honda' },
  { bike_name: 'FZS FI V4', category: '150cc', manufacturer: 'Yamaha' }
]
vehicles> db.four_wheelers.find(
|   { performance: { $gt: 5 } },
|   { vehicle_name: 1, category: 1, manufacturer: 1, _id: 0 }
| )
[
  {
    vehicle_name: 'Swift',
    category: 'car',
    manufacturer: 'Maruti Suzuki'
  },
  {
    vehicle_name: 'Eicher Pro 2049',
    category: 'mini truck',
    manufacturer: 'Eicher'
  },
  { vehicle_name: 'Creta', category: 'car', manufacturer: 'Hyundai' },
  { vehicle_name: 'Blaize', category: 'bus', manufacturer: 'Tata' },
  {
    vehicle_name: 'Innova Crysta',
    category: 'car',
    manufacturer: 'Toyota'
  }
]
vehicles>
```

QUESTION 2:

Use MongoDB to implement the following DB operations for a Zoo

1.Create a database called ‘animal’ and write a MongoDB query to select database as ‘animal’.

2.Write a MongoDB query to display all the databases.

3. Create a collection called ‘wild_animals’.(use capping) and Create a collection called ‘domestic_animals’.

```
For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-03-11T14:04:27.570+05:30: Access control is not enabled for the database. Read and write acces
s to data and configuration is unrestricted
------

Warning: Found ~/.mongorc.js, but not ~/.mongoshrc.js. ~/.mongorc.js will not be loaded.
  You may want to copy or rename ~/.mongorc.js to ~/.mongoshrc.js.
test> use animal
switched to db animal
animal> show dbs
admin      40.00 KiB
config    108.00 KiB
local      80.00 KiB
vehicles   80.00 KiB
animal> db.createCollection("wild_animals", { capped: true, size: 1048576, max: 1000 })
{ ok: 1 }
animal> db.createCollection("domestic_animals")
{ ok: 1 }
```

4. Add 5 wild_animal details to the collection named ‘wild_animals’. Each document consists of following fields as animal_name, nature (harm or harmless), favorite_foods (meat, rabbits, deer etc) as array, care_taker_name, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

```
animal> db.wild_animals.insertMany([
|   {
|     animal_name: "Lion",
|     nature: "harm",
|     favorite_foods: ["meat", "deer"],
|     care_taker_name: "Rajesh",
|     life_span: 12,
|     timestamp: new ISODate("2026-01-15"),
|     expenses: 50000
|   },
|   {
|     animal_name: "Tiger",
|     nature: "harm",
|     favorite_foods: ["meat", "rabbits"],
|     care_taker_name: "Priya",
|     life_span: 15,
|     timestamp: new ISODate("2026-02-01"),
|     expenses: 60000
|   },
|   {
|     animal_name: "Elephant",
|     nature: "harmless",
|     favorite_foods: ["grass", "leaves"],
|     care_taker_name: "Kumar",
|     life_span: 60,
|     timestamp: new ISODate("2026-01-10"),
|     expenses: 80000
|   },
|   {
|     animal_name: "Bear",
|     nature: "harm",
|     favorite_foods: ["meat", "fish"],
|     care_taker_name: "Suresh",
|     life_span: 25,
|     timestamp: new ISODate("2026-02-20"),
|     expenses: 45000
|   },
|   {
|     animal_name: "Eagle",
|     nature: "harmless",
|     favorite_foods: ["rabbits", "fish"],
|     care_taker_name: "Anita",
|     life_span: 30,
|     timestamp: new ISODate("2026-03-01"),
|     expenses: 30000
|   }
| ])
|
{
  acknowledged: true,
```

```
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b13d6b56df15d7eb7c2907'),
    '1': ObjectId('69b13d6b56df15d7eb7c2908'),
    '2': ObjectId('69b13d6b56df15d7eb7c2909'),
    '3': ObjectId('69b13d6b56df15d7eb7c290a'),
    '4': ObjectId('69b13d6b56df15d7eb7c290b')
  }
}
```

5. Add 5 domestic-animal details to the collection named ‘domestic_animals’. Each document consists of following fields as animal_name, gender (male or female),favorite_foods (meat, rabbits, deer etc) as array, animal_petname, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

```
animal> db.domestic_animals.insertMany([
|   {
|     animal_name: "Dog",
|     gender: "male",
|     favorite_foods: ["meat", "bones"],
|     animal_petname: "Bruno",
|     life_span: 12,
|     timestamp: new ISODate("2026-01-20"),
|     expenses: 15000
|   },
|   {
|     animal_name: "Cat",
|     gender: "female",
|     favorite_foods: ["fish", "milk"],
|     animal_petname: "Whiskers",
|     life_span: 15,
|     timestamp: new ISODate("2026-02-05"),
|     expenses: 12000
|   },
|   {
|     animal_name: "Cow",
|     gender: "female",
|     favorite_foods: ["grass", "hay"],
|     animal_petname: "Bessie",
|     life_span: 20,
|     timestamp: new ISODate("2026-01-12"),
|     expenses: 25000
|   },
|   {
|     animal_name: "Horse",
|     gender: "male",
|     favorite_foods: ["grass", "oats"],
|     animal_petname: "Thunder",
|     life_span: 25,
|     timestamp: new ISODate("2026-02-15"),
|     expenses: 35000
|   },
|   {
|     animal_name: "Rabbit",
|     gender: "female",
|     favorite_foods: ["carrots", "grass"],
|     animal_petname: "Bunny",
|     life_span: 8,
|     timestamp: new ISODate("2026-03-05"),
|     expenses: 8000
|   }
| ])
|
{
```

```
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b13d7256df15d7eb7c290c'),
    '1': ObjectId('69b13d7256df15d7eb7c290d'),
    '2': ObjectId('69b13d7256df15d7eb7c290e'),
    '3': ObjectId('69b13d7256df15d7eb7c290f'),
    '4': ObjectId('69b13d7256df15d7eb7c2910')
  }
}
```

6. Write a MongoDB query to display all documents available in wild_animals and domastic_animals.

```
animal> db.wild_animals.find().pretty()
[
  {
    _id: ObjectId('69b13d6b56df15d7eb7c2907'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [ 'meat', 'deer' ],
    care_taker_name: 'Rajesh',
    life_span: 12,
    timestamp: ISODate('2026-01-15T00:00:00.000Z'),
    expenses: 50000
  },
  {
    _id: ObjectId('69b13d6b56df15d7eb7c2908'),
    animal_name: 'Tiger',
    nature: 'harm',
    favorite_foods: [ 'meat', 'rabbits' ],
    care_taker_name: 'Priya',
    life_span: 15,
    timestamp: ISODate('2026-02-01T00:00:00.000Z'),
    expenses: 60000
  },
  {
    _id: ObjectId('69b13d6b56df15d7eb7c2909'),
    animal_name: 'Elephant',
    nature: 'harmless',
    favorite_foods: [ 'grass', 'leaves' ],
    care_taker_name: 'Kumar',
    life_span: 60,
    timestamp: ISODate('2026-01-10T00:00:00.000Z'),
    expenses: 80000
  },
  {
    _id: ObjectId('69b13d6b56df15d7eb7c290a'),
    animal_name: 'Bear',
    nature: 'harm',
    favorite_foods: [ 'meat', 'fish' ],
    care_taker_name: 'Suresh',
    life_span: 25,
    timestamp: ISODate('2026-02-20T00:00:00.000Z'),
    expenses: 45000
  },
```

```
  {
    _id: ObjectId('69b13d6b56df15d7eb7c290b'),
    animal_name: 'Eagle',
    nature: 'harmless',
    favorite_foods: [ 'rabbits', 'fish' ],
    care_taker_name: 'Anita',
    life_span: 30,
    timestamp: ISODate('2026-03-01T00:00:00.000Z'),
    expenses: 30000
  }
]
```

```
animal> db.domestic_animals.find().pretty()
[
  {
    _id: ObjectId('69b13d7256df15d7eb7c290c'),
    animal_name: 'Dog',
    gender: 'male',
    favorite_foods: [ 'meat', 'bones' ],
    animal_petname: 'Bruno',
    life_span: 12,
    timestamp: ISODate('2026-01-20T00:00:00.000Z'),
    expenses: 15000
  },
  {
    _id: ObjectId('69b13d7256df15d7eb7c290d'),
    animal_name: 'Cat',
    gender: 'female',
    favorite_foods: [ 'fish', 'milk' ],
    animal_petname: 'Whiskers',
    life_span: 15,
    timestamp: ISODate('2026-02-05T00:00:00.000Z'),
    expenses: 12000
  },
  {
    _id: ObjectId('69b13d7256df15d7eb7c290e'),
    animal_name: 'Cow',
    gender: 'female',
    favorite_foods: [ 'grass', 'hay' ],
    animal_petname: 'Bessie',
    life_span: 20,
    timestamp: ISODate('2026-01-12T00:00:00.000Z'),
    expenses: 25000
  },
  {
    _id: ObjectId('69b13d7256df15d7eb7c290f'),
    animal_name: 'Horse',
    gender: 'male',
    favorite_foods: [ 'grass', 'oats' ],
    animal_petname: 'Thunder',
    life_span: 25,
    timestamp: ISODate('2026-02-15T00:00:00.000Z'),
    expenses: 35000
  },
```

```
  {
    _id: ObjectId('69b13d7256df15d7eb7c2910'),
    animal_name: 'Rabbit',
    gender: 'female',
    favorite_foods: [ 'carrots', 'grass' ],
    animal_petname: 'Bunny',
    life_span: 8,
    timestamp: ISODate('2026-03-05T00:00:00.000Z'),
    expenses: 8000
  }
]
```

7. Write a MongoDB query to display only animal name and expenses in all the collection of the database

```
animal> db.domestic_animals.find(
| {},
| { animal_name: 1, expenses: 1, _id: 0 }
| )
[
  { animal_name: 'Dog', expenses: 15000 },
  { animal_name: 'Cat', expenses: 12000 },
  { animal_name: 'Cow', expenses: 25000 },
  { animal_name: 'Horse', expenses: 35000 },
  { animal_name: 'Rabbit', expenses: 8000 }
]
animal> db.wild_animals.find(
| {},
| { animal_name: 1, expenses: 1, _id: 0 }
| )
[
  { animal_name: 'Lion', expenses: 50000 },
  { animal_name: 'Tiger', expenses: 60000 },
  { animal_name: 'Elephant', expenses: 80000 },
  { animal_name: 'Bear', expenses: 45000 },
  { animal_name: 'Eagle', expenses: 30000 }
]
animal>
```

8. Write a MongoDB query to display domestic_animals whose life is a particular year

```
animal> db.domestic_animals.find(
| { life_span: 12 }
| )
[
  {
    _id: ObjectId('69b15fcb00f5c5b8857c2907'),
    animal_name: 'Dog',
    gender: 'male',
    favorite_foods: [ 'meat', 'bones' ],
    animal_petname: 'Bruno',
    life_span: 12,
    timestamp: ISODate('2026-01-20T00:00:00.000Z'),
    expenses: 15000
  }
]
animal> db.domestic_animals.find(
| { life_span: 12 },
| { animal_name: 1, life_span: 1, _id: 0 }
| )
[ { animal_name: 'Dog', life_span: 12 } ]
animal>
```

9. Write a MongoDB query to display wild_animals available under a particular care_taker

```
animal> db.wild_animals.find(
| { care_taker_name: "Rajesh" }
| )
[
  {
    _id: ObjectId('69b15fd400f5c5b8857c290c'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [ 'meat', 'deer' ],
    care_taker_name: 'Rajesh',
    life_span: 12,
    timestamp: ISODate('2026-01-15T00:00:00.000Z'),
    expenses: 50000
  }
]
animal> db.wild_animals.find(
| { care_taker_name: "Rajesh" },
| { animal_name: 1, care_taker_name: 1, _id: 0 }
| )
[ { animal_name: 'Lion', care_taker_name: 'Rajesh' } ]
animal>
```

10.Write a MongoDB query to display animal name, favorite_foods and expenses details whose lifespan is more than 5 years.

```
animal> db.wild_animals.find(
|   { life_span: { $gt: 5 } },
|   { animal_name: 1, favorite_foods: 1, expenses: 1, _id: 0 }
| )
[
  {
    animal_name: 'Lion',
    favorite_foods: [ 'meat', 'deer' ],
    expenses: 50000
  },
  {
    animal_name: 'Tiger',
    favorite_foods: [ 'meat', 'rabbits' ],
    expenses: 60000
  },
  {
    animal_name: 'Elephant',
    favorite_foods: [ 'grass', 'leaves' ],
    expenses: 80000
  },
  {
    animal_name: 'Bear',
    favorite_foods: [ 'meat', 'fish' ],
    expenses: 45000
  },
  {
    animal_name: 'Eagle',
    favorite_foods: [ 'rabbits', 'fish' ],
    expenses: 30000
  }
]
```

```
animal> db.domestic_animals.find(
|   { life_span: { $gt: 5 } },
|   { animal_name: 1, favorite_foods: 1, expenses: 1, _id: 0 }
| )
[
  {
    animal_name: 'Dog',
    favorite_foods: [ 'meat', 'bones' ],
    expenses: 15000
  },
  {
    animal_name: 'Cat',
    favorite_foods: [ 'fish', 'milk' ],
    expenses: 12000
  },
  {
    animal_name: 'Cow',
    favorite_foods: [ 'grass', 'hay' ],
    expenses: 25000
  },
  {
    animal_name: 'Horse',
    favorite_foods: [ 'grass', 'oats' ],
    expenses: 35000
  },
  {
    animal_name: 'Rabbit',
    favorite_foods: [ 'carrots', 'grass' ],
    expenses: 8000
  }
]
```